MODULE

05

Rebase & rewriting history

Clean, linear history: rebase, squash, reword and cherry-pick, plus the golden rule.

YOU WILL LEARN

rebase basics	`01_rebase_basics.py`
Interactive rebase	`02_interactive_rebase.py`
cherry-pick	`03_cherry_pick.py`
Exercise: clean up a branch	`ex05_clean_history.py`
Interview Q&A	`INTERVIEW.md`

WHY REBASE

Why rewrite history?

THE PROBLEM

Your branch is 3 days behind main and has commits called **”wip”**, **”fix”**, **”fix again”**. Merging it adds noise and a tangled graph that’s hard to review, bisect or revert.

THE IDEA

Before sharing, rebase onto the latest main and squash the noise: reviewers see a few clear commits in a straight line.

PICTURE IT REBASE

Re-planting your branch

🌱 **merge** ties two branches together with a knot (a merge commit). **rebase** cuts your commits off and **re-plants** them on the newest tip of main: same changes, new commits, straight line.



TRY IT REBASE

Replay the feature on the latest main

```
$ git log --oneline --graph --all
* ef6f46b Add footer
| * 572f72a Add search results
| * 92be44f Add search box
|/
* 4397492 Add home page
$ git switch -q feature/search
$ git rebase main
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/search.
$ git log --oneline --graph --all
* c5e801a Add search results
* c64e63a Add search box
* ef6f46b Add footer
* 4397492 Add home page
```

1. Before: two lines of history.
2. After: one straight line.
3. The feature commits have **new IDs**.

STEP BY STEP REBASE

merge vs rebase

	MERGE	REBASE
History	true history, with merge commits	linear, as if you started later
Rewrites commits?	no	**yes**: new IDs
Safe on shared branches?	yes	**no**: only your own commits
Conflicts	once, at the merge	possibly once per replayed commit
Typical use	merging a PR into main	updating your feature branch before a PR

> **Golden rule**: never rebase commits that other people already pulled.

TRY IT INTERACTIVE REBASE

git rebase -i: edit your history like a draft

```
# git rebase -i HEAD~3   opens this todo list:
pick  8510441 Add search
fixup 0a937f5 WIP
fixup 4f5af1c fix typo

# pick   = keep          reword = new message
# squash = melt + keep the message
# fixup  = melt + drop the message
# drop   = delete        (reorder lines to reorder commits)
```

1. Edit the words, save, close: Git rewrites the commits.
2. Oldest commit at the TOP.
3. Abort any time: **git rebase --abort**.

TRY IT INTERACTIVE REBASE

Three commits become one

```
$ git log --oneline
4f5af1c fix typo
0a937f5 WIP
8510441 Add search
04f6802 Add shop
$ git rebase -i HEAD~3
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/main.
$ git log --oneline
2799796 Add search
04f6802 Add shop
```

1. Here a tiny script plays you editing the todo list.
2. fixup + fixup = one clean **Add search** commit.
3. Shortcut: **git commit --fixup X** + **git rebase -i --autosquash**.

TRY IT CHERRY-PICK

Copy ONE commit to another branch

```
$ git log --oneline --all
3bbabdc Fix rounding in cart total
d69c0f5 Add new checkout
8f651f1 Add cart
$ git switch -q release/1.0
$ git cherry-pick -x main
[release/1.0 e4a7265] Fix rounding in cart total
 Date: Tue Oct 27 08:07:00 2026 +0000
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
e4a7265 Fix rounding in cart total
8f651f1 Add cart
$ git ls-files
cart.py
```

1. Only the fix comes over, not the unreleased feature.
2. **-x** records the original commit ID.
3. Typical: backport a hotfix to a release branch.

COMMON MISTAKE THE GOLDEN RULE

Rebasing a shared branch

✗ WRONG

```
git switch main
git rebase feature/x
git push --force
# everyone who pulled main is now broken
```

Rebase creates new commits. Teammates still have the old ones, and their next pull duplicates or loses work.

✓ RIGHT

```
git switch feature/x       # YOUR branch
git rebase origin/main
git push --force-with-lease
# main is only ever merged into
```

Rebase your **own** branch onto main; never rebase main or any shared branch.

IN REAL JOBS REBASE

Rebase at work

Update your branch before a PR

```
git fetch
git rebase origin/main
```

Address review comments neatly

```
git commit --fixup a1b2c3d
git rebase -i --autosquash origin/main
```

Avoid merge noise when pulling

```
git config --global pull.rebase true
```

Backport a fix

```
git switch release/2.3
git cherry-pick -x a1b2c3d
```

INTERVIEW SAY THIS IN THE INTERVIEW

“merge vs rebase: which do you use, and when?”

- **merge** preserves the true history with a merge commit: safe for shared branches.
- **rebase** replays my commits on a new base for a **linear** history, but rewrites them (new IDs).
- I rebase **my own** feature branch onto main before a PR, and merge or squash-merge the PR into main.
- Golden rule: never rebase commits others have pulled.

> **Follow-up they ask next:** ”How do you squash commits?” (git rebase -i with squash/fixup, or git reset --soft base + one commit, or Squash and merge on GitHub).

RECAP REMEMBER THESE

Module 5 in 5 lines

1. **rebase** = replay your commits on a new base: linear history.
2. Conflict while rebasing: fix, **add**, **rebase --continue**.
3. **rebase -i**: pick, reword, squash, fixup, drop, reorder.
4. **cherry-pick** copies single commits (hotfix backports).
5. Never rebase shared history.

PRACTICE NOW

Do exercises/ex05_clean_history.py: squash and rebase a messy branch.